LAB SIX - Transport Layer Protocols: UDP & TCP

This lab explores the operation of the Transmission Control Protocol (TCP) and the User Datagram Protocol (UDP), the two transport protocols of the Internet protocol architecture.

UDP is a simple protocol for exchanging messages from a sending application to a receiving application. UDP adds a small header to the message, and the resulting data unit is called a *UDP segment*. When a UDP segment is transmitted, the datagram is encapsulated in an IP header and delivered to its destination. There is one UDP segment for each application message.

The operation of TCP is more complex. First, TCP is a connection-oriented protocol, in which a TCP client establishes a logical connection to a TCP server before data transmission can take place. Once a connection is established, data transfer can proceed in both directions. The data unit of TCP, called a *TCP segment*, consists of a TCP header and payload that contains application data. A sending application submits data to TCP as a single stream of bytes without indicating message boundaries in the byte stream. The TCP sender decides how many bytes are put into a segment.

TCP ensures reliable delivery of data, and uses checksums, sequence numbers, acknowledgments, and timers to detect damaged or lost segments. The TCP receiver acknowledges the receipt of data by sending an acknowledgement segment (ACK). Multiple TCP segments can be acknowledged in a single ACK (cumulative ACK). When a TCP sender does not receive an ACK, the data is assumed lost and is retransmitted.

TCP has two mechanisms that control the amount of data that a TCP sender can transmit. First, the TCP receiver informs the TCP sender how much data the TCP sender can transmit, This is called *flow control*. Second, when the network is overloaded and TCP segments are lost, the TCP sender reduces the rate at which it transmits traffic. This is called *congestion control*.

The lab covers the main features of UDP and TCP. Part 1 compares the performance of data transmissions in TCP and UDP. Part 2 explores how TCP and UDP deal with IP fragmentation. The remaining parts address important components of TCP. Part 3 explores connection management, Parts 4 explores TCP retransmissions.

This lab uses the topology as shown in Figure 5.1. The IP addresses are given in Table 5.1. Before setting up the network configuration, you will need to change the interface configuration of the Cisco 3640 Router to include a **Serial Interface** in slot 3.

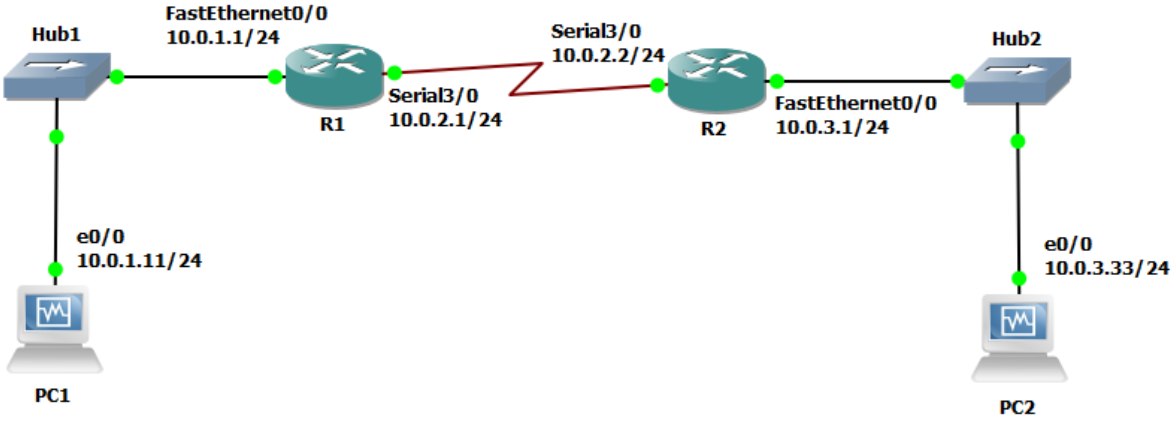

Figure 5.1 Network topology for Lab 5

PCs	eth0		Default Gateway
PC1	10.0.1.11 / 24		10.0.1.1
PC2	10.0.3.33 / 24		10.0.3.1
Routers	**FastEthernet0/0**	**Ser3/0**	**Default Gateway**
R1	10.0.1.1/24	10.0.2.1/24	10.0.2.2
R2	10.0.3.1/24	10.0.2.2/24	10.0.2.1

Table 5.1 IP addresses of the PCs and Routers

PART 1. Using iperf3 and telnet on PCs

In this lab we will use `iperf3` to test TCP and UDP transmissions. We will also use `telnet` for TCP connections. The following tables summarize the main uses of `iperf3` and `telnet`.

```
NAME
      iperf3 - perform network throughput tests

SYNOPSIS
      iperf3 -s [ options ]
      iperf3 -c server [ options ]

DESCRIPTION
      iperf3 is a tool for performing network throughput measurements. It
      can test either TCP or UDP throughput. To perform an iperf3 test the
      user must establish both a server and a client.

GENERAL OPTIONS
      -p, --port n
            set server port to listen on/connect to n (default 5201)
```

```
      -i, --interval n
            pause n seconds between periodic bandwidth reports; default is
            1, use 0 to disable
      -V, --verbose
            give more detailed output
      -J, --json
            output in JSON format
      -h, --help
            show a help synopsis

SERVER SPECIFIC OPTIONS
      -s, --server
            run in server mode
      -D, --daemon
            run the server in background as a daemon
      -1, --one -off
            handle one client connection, then exit

CLIENT SPECIFIC OPTIONS
      -c, --client host
            run in client mode, connecting to the specified server
      -u, --udp
            use UDP rather than TCP
      -b, --bandwidth n[KM]
            set target bandwidth to n bits/sec (default 1 Mbit/sec for UDP,
            unlimited for TCP). If there are multiple streams (-P flag),
            the bandwidth limit is applied separately to each stream. You
            can also add a '/' and a number to the bandwidth specifier.
            This is called "burst mode". It will send the given number of
            packets without pausing, even if that temporarily exceeds the
            specified bandwidth limit. Setting the target bandwidth to 0
            will disable bandwidth limits (particularly useful for UDP
            tests)
      -t, --time n
            time in seconds to transmit for (default 10 secs)
      -n, --bytes n[KM]
            number of bytes to transmit (instead of -t)
      -l, --length n[KM]
            length of buffer to read or write (default 128 KB for TCP, 8KB
            for UDP)
      -w, --window n[KM]
            window size / socket buffer size (this gets sent to the server
            and used on that side too)
      -M, --set-mss n
            set TCP maximum segment size (MTU - 40 bytes)

FOR MORE INFORMATION
      libiperf(3), http://software.es.net/iperf
```

NAME:

telnet – remote login to host

SYNOPSIS:

telnet **[options] [hostname] [port]**

OPTIONS:

```
-e, --exec <command>
      executes the given command
-l, --listen
      bind and listen for incoming connections
-k, --keep-open
      accept multiple connections in listen mode
-u, --udp
      use UDP instead of the default TCP
```

Please note that `iperf3` opens two different channels: a control channel and a data channel. The `iperf3` command utilizes the control channel to configure the data stream, negotiate test parameters, and exchange results. In Figure 5.2, we show an example of this behavior. Observe how `iperf3` opened a control channel with TCP port number 41196. And `iperf3` established a data channel with TCP port number 41198 to transmit the data stream. For the UDP iperf3 connection you will also note that two UDP packets are exchanged on the data channel, one from client to server and other from server to client, each carrying 4 bytes of data, before the actual user UDP data stream is initiated.

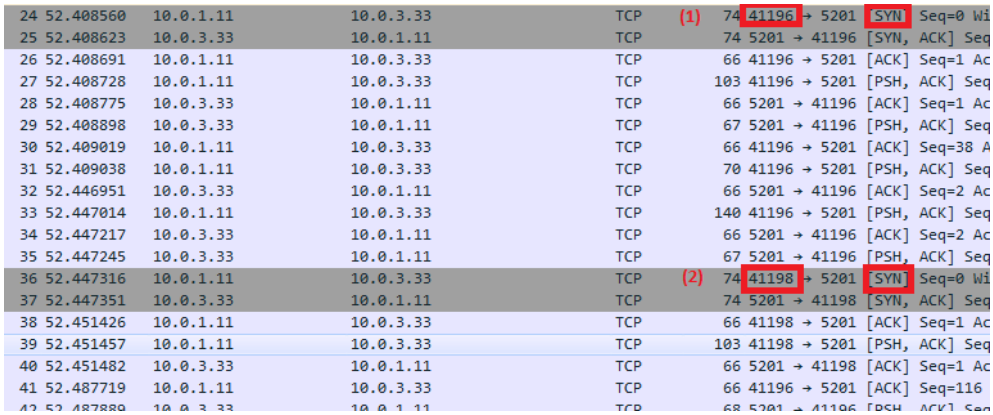

Figure 5.2 iperf control and data channels

Exercise 1(A) Cisco Router Setup with Serial Interface

1. Do this before you start your project - do not drag any routers to the project screen before configuring the serial interface on the router image as shown below.

2. Go to:
   - **Windows**: Edit -> Preferences
   - **Mac**: GNS3 -> Preferences

3. In the left-hand pane, click on the arrow next to “Dynamips”, then click on the sub-menu “IOS routers” and click “Edit” as shown in Figure 5.3 below.

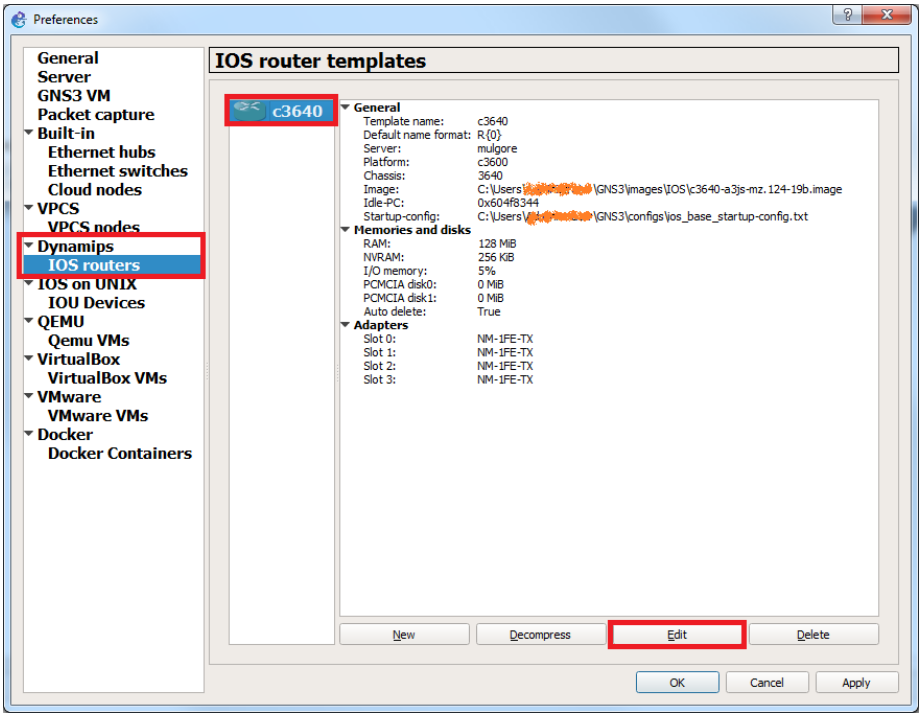

Figure 5.3 Router Interface Setup

4. Then click on Slots as shown in Figure 5.4 below. Select slot 3 under Adapters and from the dropdown menu select NM-4T as highlighted in the figure.

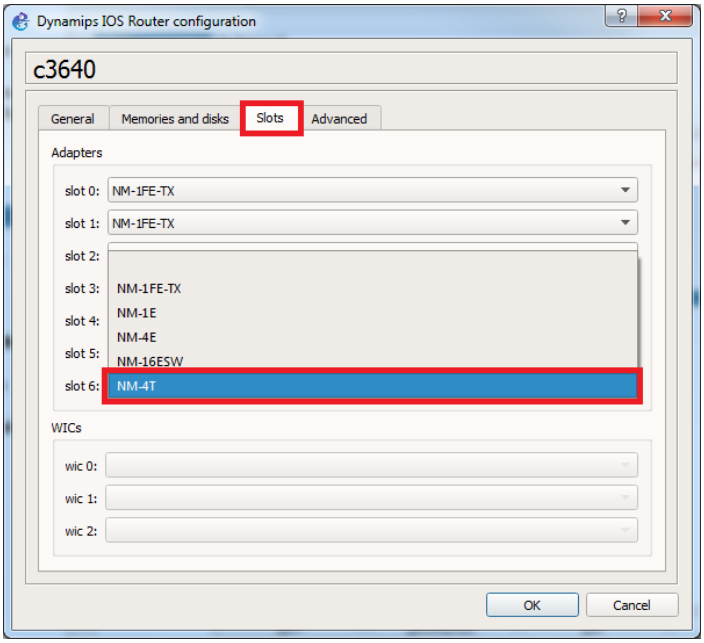

Figure 5.4 Slot Allocation

5. You should see the following. Click OK.

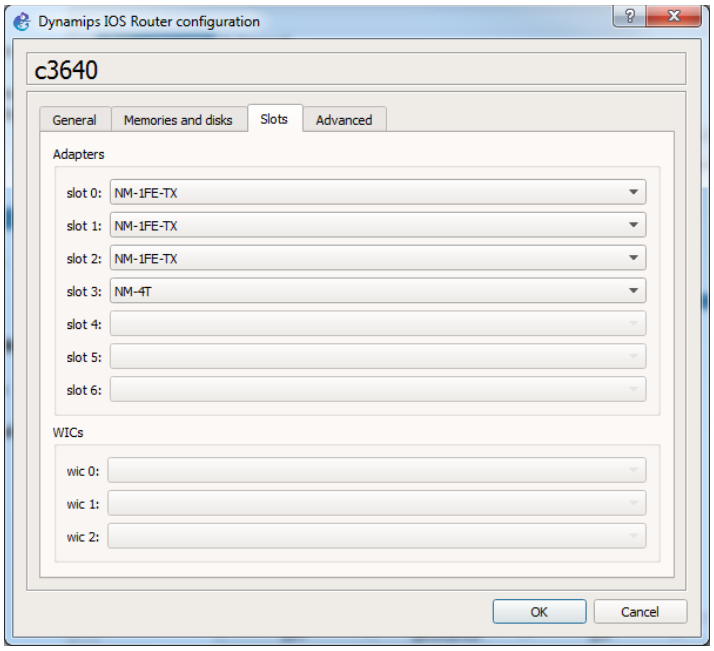

Figure 5.5

6. Then in the next screen, click Apply then OK. Your router will now have a serial interface with 4 links (Serial3/0-3).

Exercise 1(B). Network setup

1. Connect the Ethernet interfaces of the PCs as shown in Figure 5.1. Configure the IP addresses of the interfaces and default gateways as given in Table 5.1.

2. Connect the routers as shown and with a serial link between R1 and R2. Configure the IP addresses of the interfaces and default gateways as given in Table 5.1. Below shows how to configure the interface on R1.

```
R1(config)# interface Serial3/0
R1(config-if)# shutdown
R1(config-if)# ip address 10.0.2.1 255.255.255.0
R1(config-if)# no shutdown
```

3. Verify that the setup is correct by issuing a `ping` command from PC1 to PC2.

```
PC1% ping 10.0.3.33 –c 5

PC2% ping 10.0.1.11 –c 5
```

Exercise 1(C). Transmitting data with TCP

1. Start Wireshark on Hub1 connected to PC1 to capture the traffic exchanged.

2. Start the `iperf3` receiving command on PC2 (i.e., PC2 is the server) so that you will be able to receive packets being sent from PC1 (i.e., PC1 is the client).

   ```
   PC2% iperf3 -s
   ```

3. Transmit TCP packets from PC1 to PC2. Note that we are going to slow down the transmission rate of the sender by using the "`-b`" option. The underlying TCP data connection that is streaming the TCP traffic has problems keeping up with the data stream at high transmission rates resulting in a connection RESET.

   ```
   PC1% iperf3 -c 10.0.3.33 -n 10K -l 1K –b 100K
   ```

4. Save the Wireshark output. Stop data capture.

5. Kill the iperf server process running on PC2:

   ```
   PC2% killall iperf3

   Or you can terminate with: Ctrl+c
   ```

Exercise 1(D) Transmitting UDP data.

1. Start Wireshark on Hub1 connected to PC1 to capture the traffic exchanged.

2. Start the `iperf3` receiving command on PC2 so that you will be able to receive packets being sent from PC1:

   ```
   PC2% iperf3 -s
   ```

3. Transmit UDP packets from PC1 to PC2. Note that we are going to slow down the transmission rate of the sender by using the "`-b`" option. The underlying TCP data connection that is streaming the UDP traffic has problems keeping up with the data stream at high transmission rates resulting in a connection RESET.

   ```
   PC1% iperf3 -c 10.0.3.33 -u -n 10K -l 1K –b 100K
   ```

4. Save the Wireshark output. Stop data capture.

Lab Questions:

Use the data captured with Wireshark to answer the following questions.

- How many packets are exchanged in the data transfer? What are the sizes of the TCP segment?
- What is the range of the sequence numbers?
- How many packets are transmitted by PC1, and how many packets are transmitted by PC2?
- How many packets do not carry a payload, that is, how many packets are control packets?
- Compare the total number of bytes transmitted, in both directions, including Ethernet, IP, and TCP headers, to the amount of application data transmitted.

- Inspect the fields in the TCP headers. Which packets contain flags in the TCP header? Which types of flags do you observe?
- Compare the amount of data transmitted in the TCP and the UDP data transfers.
- Take the largest UDP segment and the largest TCP segment that you observed, and compare the amount of application data that is transmitted in the UDP segment and the TCP segment.

PART 2. IP Fragmentation of UDP and TCP Traffic

In this part of the lab, you observe the effect of IP fragmentation on UDP and TCP traffic. Fragmentation occurs when the transport layer sends a packet of data to the IP layer that exceeds the Maximum Transmission Unit (MTU) of the underlying data link network. For example, in Ethernet networks, the MTU is 1500 bytes. If an IP datagram exceeds the MTU size, the IP datagram is fragmented into multiple IP datagrams, or, if the Don't Fragment (DF) flag is set in the IP header, the IP datagram is discarded and an ICMP message is sent back to the sender indicating the problem.

When an IP datagram is fragmented, its payload is split across multiple IP datagrams, each satisfying the limit imposed by the MTU. Each fragment is an independent IP datagram and is routed in the network independently from the other fragments. Fragmentation can occur at the sending host or at intermediate IP routers. Fragments are reassembled only at the destination host.

Even though IP fragmentation provides flexibility that can hide differences of data link technologies to the higher layers, it incurs considerable overhead and, therefore, should be avoided. TCP tries to avoid fragmentation with a Path MTU Discovery scheme that determines a maximum segment size (MSS), which does not result in fragmentation.

In this part, you explore the issues with IP fragmentation of TCP and UDP transmissions in the network configuration shown in Figure 5.1 and Table 5.1, with PC1 as sending host, PC2 as receiving host, and PC3 as intermediate IP router.

Exercise 2(A). UDP and Fragmentation

In this exercise you will observe IP fragmentation of UDP traffic. You will use `iperf3` to generate UDP traffic between PC1 and PC2, and gradually increase the size of UDP segment until fragmentation occurs. You will observe that IP headers do not set the DF bit for UDP payloads.

To observe the UDP segment size at which fragmentation occurs, we gradually increment the size of the UDP segment by increasing the argument given with the "`-l`" option.

1. Start Wireshark on Hub1 connected to PC1 to capture the traffic exchanged.

2. Repeat **Exercise 1(D)** with an initial segment size of "`-l 256`" and then gradually increasing segment sizes e.g., 512, 1024, 2048…, until you have observed segmentation.

3. Stop the traffic capture and save the Wireshark output.

Lab Questions:

- Determine the exact UDP segment size at which fragmentation occurs.
- Determine the UDP header size.
- Determine the maximum size of the UDP segment that the system can send and receive, regardless of fragmentation (i.e. fragmentation of data segments occurs until a point beyond which the segment size is too large to be handled by UDP/IP).

- From the saved Wireshark data, select one IP datagram that is fragmented. Look at the complete datagram before fragmentation and include all fragments after fragmentation. For each fragment of this datagram, determine the values of the fields in the IP header that are used for fragmentation (Identification, Fragment Offset, Don't Fragment Bit, More Fragments Bit).

Exercise 2(B). TCP and Fragmentation

TCP tries to completely avoid fragmentation with the following two mechanisms:

- When a TCP connection is established, it negotiates the maximum segment size (MSS) to be used. Both the TCP client and the TCP server send the MSS as an option in the TCP header of the first transmitted TCP segment. Each side sets the MSS so that no fragmentation occurs at the outgoing network interface, when it transmits segments. The smaller value is adopted as the MSS value for the connection.

- The exchange of the MSS addresses MTU constraints only at the hosts, not at the intermediate routers. To determine the smallest MTU on the path from the sender to the receiver, TCP employs a method known as Path MTU Discovery, which works as follows. The sender always sets the DF bit in all IP datagrams. When a router needs to fragment an IP packet with the DF bit set, it discards the packet and generates an ICMP error message of type "destination unreachable; fragmentation needed". Upon receiving such an ICMP error message, the TCP sender reduces the segment size. This continues until a segment size that does not trigger an ICMP error message is determined.

1. Modify the MTU of the interfaces with the values shown in Table 5.2.

Routers	**MTU size on Serial3/0**
R1	500
R2	500

Table 5.2. MTU sizes.

In Cisco IOS, you can view the MTU values of all interfaces using the `show interfaces` command. For example, on **R1**, you type

```
R1> enable
R1# show interfaces
```

The command to modify the MTU value is as follows

```
R1# configure terminal
R1(config)# interface Serial3/0
R1(config-if)# mtu 500
```

2. Make sure that MTU Path Discovery is activated (MTU probing) on PC1 and PC2.

You can check if probing is set by using this command. If the value is “0” it is disabled, if “1” it is disabled by default, and enabled when an ICMP blackhole is detected, and if “2” it is always enabled.

```
PC% sysctl net.ipv4.tcp_mtu_probing
```

Enable MTU probing on PC1 and PC2 with the following command:

```
PC% sysctl -w net.ipv4.tcp_mtu_probing=2
```

Or

```
PC% echo "2" > '/proc/sys/net/ipv4/tcp_mtu_probing'
```

3. Make sure that probing is set (enabled) on **PC1** and **PC2**.
4. Start the `iperf3` receiving command on **PC2** so that you will be able to receive packets being sent from **PC1**:

```
PC2% iperf3 -s
```

5. Start Wireshark on Hub1 connected to **PC1** to capture the traffic exchanged
6. Transmit TCP packets for 20 seconds from **PC1** to **PC2**

```
PC1% iperf3 -c 10.0.3.33 -t 20 -l 2K –b 100K
```

7. When done with this exercise, reset the MTU value to 1500 on Serial3/0 interface of both routers.

Lab Questions:

- Do you observe fragmentation? If so, where does it occur? Explain your observation.
- If you observe ICMP error messages, describe how they are used for Path MTU Discovery. Look at the first TCP segment that is sent after PC1 has received the ICMP error message. Note the segment size.

PART 3. TCP CONNECTION MANAGEMENT

TCP is a connection-oriented protocol. The establishment of a TCP connection is initiated when a TCP client sends a request for a connection to a TCP server. The TCP server must be running when the connection request is issued.

TCP requires three packets to open a connection. This procedure is called a three-way handshake. During the handshake the TCP client and TCP server negotiate essential parameters of the TCP connection, including the initial sequence numbers, the maximum segment size and the size of the windows for the sliding window flow control. TCP requires three or four packets to close a connection. Each end of the connection can be closed separately, requiring 4 packets. This is called a half-close on each side. If both sides close at the same time, then the FIN packet and the ACK can be combined and transmitted in the same segment, giving rise to only 3 packets for closing.

TCP does not have separate control packets for opening and closing connections. Instead, TCP uses bit flags in the TCP header to indicate that a TCP header carries control information. The flags involved in the opening and the closing of a connection are SYN, ACK, and FIN.

Here, you will use Telnet to set up a TCP connection and observe the control packets that establish and terminate a TCP connection. We still use the same configuration as shown in Figure 5.1 and Table 5.1.

Exercise 3(A). Opening and closing a TCP connection

Set up a TCP connection and observe the packets that open and close the connection. Determine how the parameters of a TCP connection are negotiated between the TCP client and the TCP server.

1. This part of the lab uses **PC1** and **R1** set up as in the network configuration shown in Figure 5.1.

2. Verify that the MTU values of all interfaces on the routers are set to 1500 bytes, which is the default MTU for Ethernet and Serial interfaces.

3. Create a default username "**user**" and password "**password**" on **R1**.

```
R1# config terminal
R1(config)# username user secret password
```

4. Enable a Telnet Server on **R1**.

```
R1# configure terminal
R1(config-if)# line vty 0 15
R1(config-line)# login local
R1(config-line)# end
```

5. Start Wireshark on Hub1 connected to **PC1** to capture the traffic exchanged.

6. **Establishing a TCP connection**:

On **PC1**, establish a Telnet session to **R1** with the default username ***user*** and password ***password*** using the command below:

```
PC1% telnet 10.0.1.1
```

7. **Closing a TCP connection (initiated by client):**

   On PC1, type `exit` at the Telnet prompt to terminate the connection.

```
PC1% exit
```

8. Terminate Wireshark traffic capture and save the output

Lab Questions:

Analyze the TCP segments of the transmitted packets during connection set up:

- Identify the packets of the three-way handshake. Which flags are set in the TCP headers? Explain how these flags are interpreted by the receiving TCP server and TCP client.
- During the connection setup, the TCP client and TCP server tell each other the initial sequence number (ISN#) they will use for data transmission. What are the initial sequence numbers of the TCP client and the TCP server?
- Identify the first packet that contains application data. What is the sequence number used in the first byte of application data sent from the TCP client to the TCP server?
- The TCP client and TCP server exchange window sizes to get the maximum amount of data that the other side can send at any time. Determine the values of the window sizes for the TCP client and the TCP server.
- What is the MSS value that is negotiated between the TCP client and the TCP server?
- Describe the closing process of the TCP connection?

Analyze the TCP segments of the transmitted packets during connection tear down:

- Identify the packets that are involved in closing the TCP connection. Which flags are set in these packets? Explain how these flags are interpreted by the receiving TCP server and TCP client. How many transmissions were involved in the tear down?

Exercise 3(B). Requesting a connection to a non-existing host

Observing how a TCP client tries to establish a connection to a host that does not exist.

1. Start Wireshark on Hub1 connected to **PC1** to capture the traffic exchanged.

2. Set a static entry in the ARP table of **PC1** for a non existing IP address **`10.0.1.100`**:

```
PC1% arp –s 10.0.1.100 00:01:02:03:04:05
```

3. From **PC1**, establish a Telnet session to the non-existing host:

```
PC1% telnet 10.0.1.100
```

4. Terminate the traffic capture on **PC1** and save the output.

Lab Questions:

- How often does the TCP client try to establish a connection? How much time elapses between repeated attempts to open a connection?
- Does the TCP client terminate or reset the connection when it gives up trying to establish a connection?
- Why does this experiment require setting a static ARP table entry?

Exercise 3(C). Requesting a connection to a non-existing port

When a host tries to establish a TCP connection to a port at a remote server, and no TCP server is listening on that port, the remote host terminates the TCP connection. This is observed in the following exercise.

1. Start Wireshark on Hub1 connected to **PC1** to capture the traffic exchanged.

2. Establish a TCP connection to port 80 of **R1**. Note that there is no TCP server running on **R1** that is listening on this port number:

```
PC1% telnet 10.0.1.1 80
```

3. Terminate Wireshark on **PC1** and save the output.

Lab Questions:

- How does TCP at the remote host close this connection?
- How long does the process of ending the connection take?

PART 4. RETRANSMISSIONS IN TCP

Next you observe retransmissions in TCP. TCP uses ACKs and timers to trigger retransmissions of lost segments. A TCP sender retransmits a segment when it assumes that the segment has been lost. This occurs in two situations:

- **No ACK has been received for a segment:** Each TCP sender maintains one retransmission timer for the connection. When the timer expires, the TCP sender retransmits the earliest segment that has not been acknowledged. The time is started when a segment with payload is transmitted and the timer is not running, when an ACK arrives that acknowledges new data, and when a segment is retransmitted. The timer is stopped when all outstanding data has been acknowledged.

  The retransmission timer is set to a retransmission timeout (RTT) value, which adapts to the current network delays between the sender and the receiver. A TCP connection performs round-trip measurements by calculating the delay between the transmission of a segment and the receipt of the acknowledgment for that segment. The RTT value is calculated based on these round-trip measurements. Following a heuristic called *Karn's algorithm*, measurements are not taken for retransmitted segments. Instead, when a retransmission occurs, the current RTT value is simply doubled.

- **Multiple ACKs have been received for the same segment**: A duplicate acknowledgment for a segment can be caused by an out-of-order delivery of a segment or by a lost packet. A TCP sender takes multiple, in most cases, three, duplicates as in indication that a packet has been lost. In this case, the TCP sender expedites a fast retransmit by sending an ACK for each packet that is received out of order.

A disadvantage of cumulative acknowledgments in TCP is that a TCP receiver cannot request the retransmission of specific segments. For example, if the receiver has obtained segments 1, 2, 3, 5, 6, 7 with cumulative acknowledgments the receiver can send ACKs only for segments 1, 2, 3 but not for 5, 6, 7. The problem can be remedied with an optional feature of TCP, which is called *selective acknowledgments* (SACKs). Here, in addition to acknowledging the highest sequence number of contiguous data that has been received correctly, a receiver can acknowledge additional blocks of sequence numbers. The range of these blocks is included in TCP headers as an option. Whether SACKs are used is negotiated in TCP header options when the TCP connections are created.

The exercise in this part explores aspects of TCP retransmissions that do not require access to internal timers. Unfortunately, the round-trip time measurements and the RTT values are difficult to observe and are, therefore, not included in this lab.

The network configuration for this part is the network as given in Figure 5.1 and Table 5.1.

Exercise 4(A). TCP Retransmissions

The purpose of this exercise is to observe when TCP retransmissions occur. In this part of the lab, you will transmit data from PC1 to. When you disconnect the connection to PC2, ACKs cannot reach the sending host PC1. As a result, a timeout occurs and the sender performs retransmissions.

1. When the serial interface of **R1** and **R2** are directly connected by a serial cable as in Figure 5.1, one interface functions as DCE (data circuit-terminating equipment) and the other as DTE (data terminal equipment).

   NOTE:
   We have experienced that both sides can be DCE. In IOS, a clock rate must be set on the serial interface that functions as DCE. Whether an interface functions as DCE or DTE is determined by the orientation of the serial cable. If both sides claim to be DCE, you can pick either one to change the clock rate as shown below.

2. Determine the DCE end: To check whether a cable connected to a serial interface of a router is of type DCE or DTE, type the following command:

   ```
   R1# show controllers Serial3/0
   ```

3. The command displays low-level information on the serial interface, including whether an interface functions as DTE or DCE. Look for statements “V.35 DTE cable” or “V.35 DCE cable” in the output of the commands. For example, something like the following will be displayed:

   cable type: V.11 (X.21) DCE cable, received clockrate 2015232

4. **Set the clock rate at the DCE end**:
   Once the DCE has been identified, you must set the clock rate. Assuming that the serial interface of **R1** functions as DCE (and **R2** should then be the DTE end), type the following commands:

   ```
   R1# configure terminal
   R1(config)# interface Serial3/0
   R1(config-if)# clock rate 9600
   ```

   This sets the clock rate of the serial link to 9600 bps.

   NOTE:
   Since executing the command clock rate on a DTE interface has no effect, and merely results in the display of an error message, you could ignore the above check and safely execute the command on the both **R1** and **R2**. I.e., you need not determine which interface serves as the DCE.

5. Test the link by issuing a ping from **PC1** to **PC2**. Proceed to step 6, if the ping is successful. If not, check the configuration again.

6. Start Wireshark on Hub1 connected to **PC1** to capture the traffic exchanged (set TCP filter).

7. Start the `iperf3` receiving command on **PC2** so that you will be able to receive packets being sent from PC1:

   ```
   PC2% iperf3 -s
   ```

8. Transmit TCP packets from **PC1** to **PC2** for 30 seconds with the following command:

```
PC1% iperf3 -c 10.0.3.33 -t 30
```

9. Wait for five seconds after starting `iperf3` above, then bring down **R1** Serial3/0 interface.

```
R1(config)# interface Serial3/0
R1(config-if)# shutdown
```

10. Wait for five seconds after bringing the interface down and bring the Serial interface up again.

```
R1(config)# interface Serial3/0
R1(config-if)# no shutdown
```

11. When `iperf3` is complete (30secs of transmission is done), stop Wireshark and save the captured traffic.

Lab Questions:

Analyze the Wireshark output and answer the following questions:

- When the connection is created, do the TCP sender and TCP receiver negotiate to permit SACKs? Describe the process of negotiation.
- When you first disable ip forwarding, observe the time instants when retransmissions took place. How many packets were retransmitted at one time?
- Try to derive the algorithm that sets the time when a packet is retransmitted. Use data to backup your answer. Is there a maximum time interval between retransmissions?
- After how many retransmissions, if at all, does the TCP sender stop to retransmit the segment? Describe your observations.
- After you re-enable ip forwarding in Step 5, and disable and enable ip forwarding rapidly, do you notice any difference in the retransmissions from those observed in Step 3? Specifically, do you observe fast retransmits and/or SACKs?